

Final Project Proposal

Vitamin Deficiency Risk Prediction

1. Certificate Name

Mohamed Ali Shoble

2. Project Title and Description

Title: Vitamin Deficiency Risk Prediction from Lifestyle and Environmental Factors

Vitamin deficiencies are usually only detected through lab blood tests, which are costly and not something most people get regularly. This project explores whether personal lifestyle choices and environmental factors alone — without any lab data — can predict a person's vitamin deficiency risk. Using a dataset of demographic, lifestyle, and environmental features (age, BMI, smoking, alcohol use, exercise level, diet type, sun exposure, income, and geographic latitude region), the goal is to build a regression model that estimates a continuous deficiency severity score. This could serve as an early, low-cost screening tool — flagging individuals who may benefit from further testing or dietary adjustment, before symptoms or a formal diagnosis.

3. Problem Type

Regression — the target, vitamin deficiency, is a continuous severity score (observed range in the data: 0.0 to 1.89), not a category. This is supervised learning: we train on records where the outcome score is already known.

Note on data leakage: the raw dataset also contains per-vitamin *_percent_rda columns (e.g. vitamin_d_percent_rda, vitamin_b12_percent_rda) and a symptoms_count/symptoms_list pair. Correlation analysis showed these are extremely strongly correlated with the target (e.g. vitamin_d_percent_rda: -0.81, symptoms_count: +0.71) — strong evidence the target score was derived directly from them. These columns will be excluded from the feature set to avoid leakage; the model will only use genuine lifestyle/environmental predictors, which is also a better match for the original research question (can lifestyle alone predict risk, without lab measurements?).

4. Dataset

- Source: Self-sourced CSV (train_data.csv), originally identified via a Kaggle vitamin-deficiency dataset search; already collected and in hand
- Size: 3,500 rows, 21 raw columns
- Target: vitamin deficiency — continuous deficiency severity score

Features used (lifestyle/environmental only, leakage columns excluded):

- age — numeric
- gender — categorical (Male/Female)
- bmi — numeric
- smoking_status — categorical (Never/Former/Current)
- alcohol_consumption — categorical (Moderate/Heavy/missing → treated as "None")
- exercise_level — categorical (Sedentary/Light/Moderate/Active)
- diet_type — categorical (Omnivore/Vegetarian/Vegan/Pescatarian)

- sun_exposure — categorical (Low/Moderate/High)
- income_level — categorical (Low/Middle/High)
- latitude_region — categorical (Low/Mid/High)

Excluded (leakage risk): vitamin_a_percent_rda, vitamin_c_percent_rda, vitamin_d_percent_rda, vitamin_e_percent_rda, vitamin_b12_percent_rda, folate_percent_rda, calcium_percent_rda, iron_percent_rda, symptoms_count, symptoms_list.

Preprocessing plan: fill missing alcohol_consumption values (interpreted as "None", i.e. non-drinker), one-hot/label encode categorical columns, scale numeric features (age, bmi), 80/20 train/test split.

5. Algorithms I Plan to Train

#	Algorithm	Why it fits
1	Linear Regression	Bootcamp baseline; interpretable coefficients show the direction/size of each lifestyle factor's effect on deficiency risk.
2	Decision Tree Regressor	Bootcamp model; captures non-linear thresholds (e.g. sun exposure interacting with latitude region).
3	Random Forest Regressor	Bootcamp ensemble; reduces the overfitting a single Decision Tree is prone to (observed this exact failure mode in a prior regression assignment on car prices).
4	Gradient Boosting Regressor	Researched — often strong on structured/tabular data with mixed categorical/numeric features; a good candidate for best model given the added noise from removing leakage columns.

Three (Linear Regression, Decision Tree, Random Forest) come from the bootcamp; Gradient Boosting I will research via scikit-learn documentation.

6. Evaluation Plan

Metrics for all models (on the same held-out test set):

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- R^2 (coefficient of determination)

Best-model rule: Rank all trained models by R^2 on the test set. R^2 directly answers the project's research question — how much of the variance in deficiency risk can be explained by lifestyle and environmental factors alone — which is more meaningful here than raw error magnitude, since the target is a somewhat abstract severity score rather than a real-world unit. If two models are close on R^2 , I will break the tie with lower RMSE.

I will print a comparison table with one row per algorithm and save/deploy only the winner. I expect R^2 to be noticeably lower than it would be with the leakage columns included — I will report and discuss that honestly in the paper rather than re-adding the leaking features to inflate the score.

7. Deployment Sketch

- Framework: FastAPI (simple, modern, auto-generates API docs at /docs)
- Endpoint: POST /predict

Input JSON example:

```
{
  "age": 34,
  "gender": "Female",
  "bmi": 23.5,
  "smoking_status": "Never",
  "alcohol_consumption": "Moderate",
  "exercise_level": "Light",
  "diet_type": "Vegan",
  "sun_exposure": "Low",
  "income_level": "Middle",
  "latitude_region": "High"
}
```

Output JSON example:

```
{
  "predicted_deficiency_score": 0.71,
  "risk_level": "Elevated"
}
```

(risk_level is a simple human-readable bucket derived from the numeric score, e.g. Low / Moderate / Elevated / High — for clarity, not a separate model.)

The API loads the best saved model from models/best_model.pkl plus the fitted scaler/encoders.

8. Repository Plan

```
vitamin-deficiency-risk/
├── dataset/
│   └── train_data.csv
├── src/
│   ├── preprocess.py      # cleaning, encoding, scaling, leakage-column removal,
├── train/test split
│   └── train.py           # train all 4 models, compare, save best
├── api/
│   └── app.py             # FastAPI app with /predict
├── models/
│   ├── best_model.pkl
│   └── scaler.pkl
├── notebooks/
│   └── exploration.ipynb  # optional EDA, correlation/leakage analysis
├── README.md
├── requirements.txt
└── project_paper.md
```

Run commands (planned):

```
python src/train.py      # trains models, prints comparison table, saves best
uvicorn api.app:app --reload  # starts API locally
```